

Tutorial 1: Distributed Databases Design

Question 1: Given the following relation table STUDENT (ID, NAME, AGE, SALARY) and predicates p1: SALARY > 30000, p2: SALARY < 30000.

<u>ID</u>	NAME	AGE	SALARY
1289	John	24	12000
8907	Sally	29	67050
7643	Elvin	22	51980
0988	Kelly	42	30000
6543	Emily	19	28760
0986	Sally	46	54000
2345	Thomas	23	29999

- Perform a horizontal fragmentation of the table based on the given predicates.
- Is this fragmentation correct? If no, explain why.
- Generate a correct horizontal fragmentation using *minterm* predicates.
- Someone suggests to vertically fragment this relation to S1(ID, NAME, AGE) and S2(NAME, SALARY). Is this fragmentation correct? If no, use the tuples given in the relation above to illustrate the problems.
- For the following relation STUDY(S_ID, COURSE, RESULT) where S_ID is a foreign key to STUDENT.ID, perform derived horizontal fragmentation using *semi-join*.

<u>S_ID</u>	<u>COURSE</u>	RESULT
1289	INFS1200	7
1289	INFS2200	6
8907	DECO1400	5
8907	INFS1200	4
8907	INFS2200	4
7643	COMP1002	6
0988	COMP4500	6
0988	INFS2200	5
6543	INFS1200	4
0986	INFS1200	7
2345	INFS1200	7

Question 2: A correct fragmentation needs to satisfy the following three properties:

- 1) completeness,
- 2) disjointness, and
- 3) reconstructability.

Please discuss whether each of the following fragmentation schemes meet these criteria, and how the original relation can be reconstructed.

- (a) Primary horizontal fragmentation using minterm predicates.
- (b) Derived horizontal fragmentation using semi-joins with the owner relation correctly fragmented.
- (c) Vertical fragmentation with primary key attributes fully replicated in all fragments.
- (d) Vertical fragmentation with primary key attributes replicated in only some fragments.

Question 3: After a relation R is fragmented correctly, how to insert a new tuple into R or update an existing tuple? Using primary horizontal fragmentation as an example.

Question 4: Consider relation STUDENT of question 1. Assume there are some queries:

q1: Find student names, given a certain age.

```
SELECT NAME
FROM STUDENT
WHERE AGE = Value;
```

q2: Find the IDs and names of all students.

```
SELECT ID, NAME
FROM STUDENT;
```

q3: Find the names of students within a salary range.

```
SELECT NAME, SALARY
FROM STUDENT
WHERE SALARY BETWEEN
Value1 AND Value2;
```

q4: Find salary and age information of all students.

```
SELECT NAMES, AGE, SALARY
FROM STUDENT;
```

Let $freq(q)$ be the application access frequency measure, where q is the query. Assume $freq(q_1) = 15$, $freq(q_2) = 20$, $freq(q_3) = 10$, and $freq(q_4) = 25$. Construct the Attributed Affinity Matrix based on the queries.

---ooo000OOO000ooo---